

sidestr: User-Activated Sidechains

Melvin Carvalho
melvincarvalho@gmail.com
sidestr.com

Draft, 27 September 2026

Abstract. A sidechain would allow a coin to be used on a second ledger without changing the rules of the first. Sidechains have been proposed for a decade, but each proposal has needed the parent chain to enforce the peg, and the parent chain has not changed. We propose a chain that needs nothing from its parent. Blocks are valid because they are signed, not because they were mined. The rules are the parent's, with one added and one removed: a block must carry a signature, and there is no subsidy. Every coin on the chain is a coin locked on the parent. Anyone can validate every block in full, in a browser, without a key and without permission. The signers decide the order of blocks and nothing else; a block that breaks a rule is invalid whatever signature it carries. The rules themselves are documents with activation heights, and a node applies a rule because its operator chose it. Blocks, chain tips and transactions travel as signed events over public relays, so the network needs no servers of its own. We describe the chain, the peg, the rules, the trust the reader accepts at each level, and the path by which that trust is reduced. Eight chains have run on a test network since September 2026.

1. Introduction

sidestr is a chain beside a chain. Bitcoin settles a few transactions a second for the whole world. Anything that needs more, an issued asset, a market, a game, a program, has been built either on a chain of its own, with its own coin and its own security, or on a server, with a database and an operator. The first is expensive and the second is not Bitcoin.

The usual answer is a sidechain: a second chain whose coins are Bitcoin locked on the first. Proposals differ in who holds the lock. A federation can hold it, as Liquid does, and then the sidechain is as trustworthy as the federation. The parent chain can hold it, as drivechains propose, and then the parent's rules must change, which they have not. What every proposal shares is that the sidechain's validity is checked by nodes that run the sidechain's software, and the reader of a balance trusts those nodes.

We take a different starting point. The reader validates. A chain is a signed document naming its parent, its signing challenge and its rules. Blocks are the parent's block format, produced by whoever holds the challenge key, and any node, a phone in a browser included, checks every block and every transaction under the parent's own rules. The signers order blocks. They cannot mint, cannot forge a transaction and cannot change a rule, because every reader checks. What a reader trusts is stated in the chain's document and can be read before a coin is sent.

This is a small claim. It does not make the peg trustless. It makes the trust explicit and the validation universal, and it leaves the parent chain alone. The rest of the paper describes the mechanism and what has been run.

2. Chains

A chain is an overlay on its parent's parameters. The document sets what differs and inherits everything else:

parameter	meaning
parent	the parent chain, by a short alias
challenge	a script; a block is valid when its signature satisfies it
powLimit	a proof of work cheap enough for a laptop, so a block without the key still costs something
subsidy	zero
addressPrefix	distinct from the parent's, so no parent address is ever a sidechain address
pegConfirmations	parent confirmations before a peg-in may be claimed
refundBlocks	the timelock after which an unclaimed peg returns to its owner
genesis	the pegs the chain starts from

Header format, proof-of-work hash, script rules, weight limits and the transaction sighash all follow the parent. A chain beside Bitcoin has Bitcoin's header and SHA256d. A chain beside the BLAKE2b fork has the v2 header and the unified sighash. A chain beside Litecoin would have script. Nothing in the document names a hash; the parent decides.

A parent may itself be a sidechain. Chains then form a tree. Coins flow down by peg-in and up by peg-out, and a single validator checks every level with one engine. Trust compounds with depth: a reader of a chain at depth n trusts every signer between it and the proof-of-work root for ordering and for which pegs exist. Depth is cheap for tests and agents and a reason to keep value near the root.

3. Blocks

A block is valid when it is valid under the parent's rules with three changes, checked in order.

1. The header meets powLimit. There is no difficulty adjustment.
2. The coinbase carries, after the witness commitment, a marker and a script witness that satisfies the challenge for the block's signet hash, computed as BIP 325 computes it. A block without the marker, or whose witness fails the challenge, is invalid.
3. The coinbase's outputs sum to at most the block's fees plus the peg-in claims the block makes.

Everything else, heights, previous hash, timestamps, transaction validity, is the parent's. The challenge is a script, so it can be one key, a threshold of keys, or anything the interpreter runs. Changing the challenge is a rule change.

A producer makes a block when it has transactions and otherwise idles, with a heartbeat block often enough that timelocks and maturity keep moving. Blocks are receipts for transactions, not a clock.

4. Pegs

A peg-in is a transaction on the parent that pays a taproot output the peg holder can spend and carries an OP_RETURN naming the chain and the script where the coins should appear on it. The taproot output's script path is a refund: the pegger's own key, spendable after refundBlocks. If the chain never claims the peg, or dies, the coins come back to whoever pegged them after the timelock, with no signer involved. A dead sidechain costs time, not coins.

The signer announces, with every chain tip, the script a peg-in should pay. A wallet builds a peg-in from the announcement alone, and a third party can tell the peg from the wallet's change.

After pegConfirmations, a block may claim the peg: its coinbase pays the named script the peg's amount, and the next output is an OP_RETURN naming the parent outpoint. That pairing is what lets a validator with no parent view bind each claimed amount to one outpoint. A claim of an outpoint already claimed is invalid. Supply on the chain equals the pegs claimed less the pegs burned. Nothing else mints.

A peg-out is a sidechain transaction paying a burn output: an OP_RETURN with the parent script that should be paid, and a value that leaves the chain's supply. The peg holder pays that script that value on the parent, from the peg outputs, in a transaction that names the sidechain txid. A validator with a parent view checks that every burn is paid within pegoutBlocks and publishes the ones that are not. This is the peg holder's promise and the validators' record of whether it was kept.

5. Rules

The chain's rules are documents. Each is a signed, addressable event with an activation height, published under a rule key. A node is configured with the rule keys it follows and applies a rule from its activation height because its operator chose that key. A signer who wants a rule changed publishes a document and waits to see who adopts it. A node that adopts nothing keeps the rules it has. Two nodes that adopted different documents disagree from the activation height, exactly as they would on any chain, and the block signatures do not settle it.

This is what "user activated" means. The signers can stall the chain. They cannot change it.

Rules extend the chain as well as constrain it. An issued asset, a constant-product pool between assets, a record store, an EVM with the pegged coin as gas: each is a rule a chain document names, validated by every node, with the pegged coin as the only thing called by the parent's name. An issued asset is unbacked and every document that names it says so.

6. Validation

The reader runs the parent's engine. The same code that validates a Bitcoin block validates a sidestr block, with the three checks of section 3 and the rules of section 5 added as overlays. A

validator needs no key, no hash power and no permission. It needs the chain document, the blocks and the rule documents its operator adopted.

Blocks are served as a flat file from any mirror. Chain tips are published as signed events on public relays: the height, the last twelve headers, the mirrors that serve the blocks and the peg script. A client that knows only the chain id takes the newest announcement, reads the chain document from a mirror it names, and accepts the mirror when the document's signer is the announcement's author. A mirror is then held to the announcement: it may be behind the signer but never ahead. A chain id is a name, not a proof, so a client shows the signer it settled on.

Transactions reach a producer the same way, as signed events on a relay, from anyone's key: a transaction authorises itself. A producer includes what validates. A wallet with no parent node may publish a signed parent transaction the same way, and a producer with a node broadcasts it if and only if that node's own mempool policy accepts it as it stands, never retrying a refusal. The node is the judge; the producer only carries.

A browser is a full validator. It downloads the block file, checks every block, keeps the UTXO set and its own cache of validated state, and signs transactions with a key it holds. There is no light client mode because there is nothing to be light about at these sizes: a chain of a few hundred blocks is a few megabytes, and a tab validates it from the first block.

7. Trust

We state plainly what the reader trusts, by level.

level	the reader has	and trusts the signers for
1	the sidechain rules	which peg-ins exist, and that peg-outs are paid
2	a parent view: headers and the peg outputs	nothing about pegs; still ordering
3	level 2, several independent signers, rotation, a recovery path	liveness only

At every level the reader trusts no one for validity. A block that mints, spends a coin twice or breaks a rule is invalid to every reader, and the signature does not help it.

What the signers can do is stall, reorder and censor. Stalling stops the chain, and pegs refund after the timelock. Reordering and censoring are visible to every validator, since every validator sees every block, and the remedy at level 1 is to leave. What the peg holder can do at level 1 is spend the peg outputs. The refund path limits this to coins not yet swept, and the record shows it. This is why level 1 is for coins with no value, and why the chain document says which level it is.

A mirror that lies is caught by the announcement and by validation. A parent that stalls freezes every child's refund clock with it; the coins wait.

8. Reducing the trust

Level 1 trusts the signers for which pegs exist. Level 2 removes that by giving the reader a parent view: the parent's headers, which a browser can hold, and the peg outputs, which it can check by

merkle path. A reader at level 2 refuses a claim it cannot see on the parent. The remaining trust is ordering and liveness, which several signers with a threshold challenge reduce, and rotation and a recovery path reduce further.

The parent's proof of work can be made to bound the chain's history. A producer writes its chain tip into the parent now and then, so a reader knows that the chain as of that height existed when that parent block was mined and cannot be rewritten before it. The parent does not check the tip; it timestamps it.

Beyond this the path is the one any chain validated from a snapshot must walk. A reader that starts from a UTXO snapshot trusts whoever gave it the snapshot's hash. Attestations from named keys make that trust explicit and pluggable. A commitment to the UTXO set in the coinbase, as producer policy, puts the hash under the chain's own signatures where one RPC call checks it. Fraud proofs let one honest node refute a false entry with a few hundred bytes anyone can verify against headers. A validity proof, when a prover exists at this scale, replaces "accept unless refuted" with "accept only with a proof", against the same commitment, with no change between the steps. The first three are buildable now. The fourth is the destination and the reason to lay the others in its shape.

9. What runs

Eight chains have run beside the BLAKE2b testnet4 since 15 September 2026, on one machine, and a ninth beside stock testnet4 by a third party. Between them they have made about 5,600 blocks. The first chain's genesis is four peg-ins of 25 tBTC each, made on 15 September, and reproduces its genesis block byte for byte from the peg-in records.

chain	what it tests	blocks
siding	the first chain; one signer, hourly heartbeat	332
tally	issued assets and a pool between them and the coin	852
desk	a chain that pays now for a coinbase reward locked on the parent	842
gitmark	chain tips checkpointed into the parent; git commits marked on the chain	801
evm	an EVM as a rule, the coin as gas, a public JSON-RPC	751
fed	three signers, a threshold challenge, co-signing over the relay	642
melchain, capewars-s6	a personal chain; a game season with bots as users	710, 663

Proven end to end, from a browser key, against a live parent: a peg-in built from the chain tip's announcement, relayed to a producer's node and claimed; a trade on the pool; a peg-out burned on the chain and paid on the parent; a paywall that a process verifies itself, opened by a user with an in-page signature; a wallet with one key across every chain; a third party's chain, beside stock testnet4 with a stock header and a stock sighash, opening in the same wallet.

Also found, and worth recording. The first two versions of the spec took the first taproot output as the peg, which misread a wallet's change as the peg; a third party found it on the first chain beside

stock testnet4. A hand-typed script in a genesis document put coins at an address nobody held; they are backed on the parent and will be re-pegged, but the lesson is that a script is never typed. A transaction that a public node refused was broadcast through our own node instead; the rule now is that a refusal anywhere is a refusal everywhere. None of this cost a coin on a chain with value, because no chain with value has run. That is what testnets are for.

10. Related work

Elements and Liquid are signed-block sidechains with a federated peg, and this is a small copy of them with the validation moved to the reader and the rules moved into documents. BIP 325 signet supplies the challenge mechanism unchanged. Drivechains and spacechains ask the parent to enforce the peg, which is the level this does not reach and the reason it needs no change to the parent. RGB and Taproot Assets validate on the client, as this does, but keep state with the owner rather than in blocks every reader can replay; a lost wallet loses the history. Shielded Bitcoin publishes every transfer on the parent and replays state from it, with zero-knowledge proofs for privacy; it has no blocks of its own and pays a parent fee per transfer. Rollups post state to a parent that verifies it, which Bitcoin cannot. Stellar's consensus gave us the idea that trust is chosen per node.

11. Conclusion

We have proposed a chain beside a chain that asks nothing of its parent. The parent's rules validate its blocks, a signature orders them, and every coin on it is a coin locked above. The reader checks everything, in a browser, and trusts the signers for ordering and, at the first level, for the peg, which the document says and the refund path bounds. The rules are documents chosen by each operator, so the signers cannot change the chain they order. Blocks and transactions travel as signed events over relays anyone runs. From here the trust is reduced step by step, with a parent view, more signers, checkpoints into the parent's proof of work, commitments, fraud proofs and, at the end, a validity proof, without any of the steps needing the one before to be undone. Eight chains have run for two weeks on a test network and have made about 5,600 blocks. It is enough to see the shape. It is not enough to hold value, and the paper says so.

References

1. S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008.
2. A. Back et al., "Enabling Blockchain Innovations with Pegged Sidechains," 2014.
3. K. Alm, A. Towns, "BIP 325: Signet," 2019.
4. P. Sztorc, "BIP 300: Hashrate Escrows," 2017.
5. R. Linus et al., "BitVM: Compute Anything on Bitcoin," 2023.
6. M. Orlovsky et al., "RGB: Client-side validated smart contracts," 2019–2024.
7. C. Shikhelman, M. Komarov, A. Moskvina, "Shielded Bitcoin: Private Transfers on the Bitcoin L1," 2026.
8. D. Mazières, "The Stellar Consensus Protocol," 2015.
9. sidestr specification, version 0.0.4, <https://sidestr.com/spec/>, 2026.

10. datstr specification, <https://datstr.com/spec/>, 2026.